

🔑 Objectifs de la feuille

- Les vues DJANGO
- Comprendre les templates
- Logique de programmation dans les templates
- Héritage de templates
- Mettre du style dans ses templates

Les vues : à quoi ça sert et comment ça marche ?

Petit focus sur les vues car nous avons jusqu'ici créer 3 ou 4 vues basiques sans vraiment creuser le sujet. Les vues sont la deuxième notion du concept MVC (Modèle Vue Contrôleur). Les vues créent un contexte, intègre un **template** (ou gabarit) puis retournent un objet de type **HttpResponse** . Tout d'abord votre serveur web reçoit une requête avec comme contenu: une adresse, un entête HTTP , divers données fournit par GET, POST, etc. Ensuite le serveur web demande à DJANGO ce qu'il doit faire avec cette requête. L'adresse est comparée dans une sorte de table de routage (**urls.py**) puis DJANGO exécute la fonction associée au pattern de l'adresse . Cette fonction est donc une vue, puisqu'elle reçoit un contexte et renvoie un objet **HttpResponse** . Par exemple :

- j'ai les URLs suivantes dans mon fichier **monApp/urls.py** :

```
urlpatterns = [
    path("", views.home, name="home"),
    path("home/", views.home, name="home"),
    path("about/", views.about, name="about"),
    path("contact/", views.contact, name="contact"),
    path('home/<param>', views.home, name='home'),
    path('produits/', views.ListProduits, name='produits'),
    path('categories/', views.ListCategories, name='categories'),
    path('statuts/', views.ListStatuts, name='statuts')
    path('rayons/', views.ListRayons, name='rayons')
]
```

- j'ai les vues suivantes dans mon fichier **monApp/views.py** :

```
def home(request, param=None):
    if param:
        return HttpResponse(f"<h1>Hello {param}</h1>")
    return HttpResponse("<h1>Hello Django</h1>")

def about(request):
    return HttpResponse("<h1>About Us</h1>")

def contact(request):
    return HttpResponse("<h1>Contact Us</h1>")
```

Première chose que nous avons déjà remarqué, c'est que les vues dans notre fichier `views.py` ne sont en réalité que de simples fonctions qui récupèrent un objet `request` et qui retournent un objet `HttpResponse`. En anglais `request` signifie "demande / sollicitation", ce sont donc des informations envoyées par un client et celui-ci attend une réponse en fonction des informations qu'il a envoyées. Qui a-t-il dans un objet `request` ? Pour cela, rien de plus simple, ajouter la fonction `dir` dans votre vue `home`:

```
def home(request, param=None):
    print (dir(request))
    if param:
        return HttpResponse(f"<h1>Hello {param}</h1>")
    return HttpResponse("<h1>Hello Django</h1>")
```

Le résultat est long dans le terminal... Vous voulez encore plus long ? Pour voir les données associées à l'objet `request`:

```
def home(request, param=None):
    print (request.__dict__)
    if param:
        return HttpResponse(f"<h1>Hello {param}</h1>")
    return HttpResponse("<h1>Hello Django</h1>")
```

Je ne peux pas copier coller le résultat (il y en a trop). Je vous invite donc à effectuer cette manipulation sur votre machine. Mais vous verrez qu'il existe des données sur tout comme l'adresse IP du client, le navigateur utilisé, les données de session, le port utilisé, le langage, le csrftoken, les données GET et POST, etc.

Bon passons aux choses sérieuses. Comment lire les données GET et POST envoyées par le client ?

```
def home(request, param=None):
    if 'name' in request.GET:
        string = request.GET['name']
        return HttpResponse("Bonjour %s!" % string)
    if param:
        return HttpResponse(f"<h1>Hello {param}</h1>")
    return HttpResponse("<h1>Hello Django</h1>")
```

Rendez vous à l'url [http://127.0.0.1:8000/monApp/home?name="cricri"](http://127.0.0.1:8000/monApp/home?name='cricri')

Une réponse où aucune erreur n'est à signaler possède le code 200, vous pouvez le vérifier sur votre navigateur en lisant l'entête de la réponse:

Nom	État
 home?name=%22cricri%22	200

Il est possible de retourner d'autres types de réponse, comme la fameuse 404 indiquant que le document est introuvable :

```
def home(request):
    return HttpResponseNotFound("Erreur fichier introuvable")
```

Nom	État
✖ home?name=%22cricri%22	404

Il existe d'autres réponses HTTP:

HttpResponse	→ 200
HttpResponseRedirect	→ 302
HttpResponsePermanentRedirect	→ 301
HttpResponseNotModified	→ 304
HttpResponseBadRequest	→ 400
HttpResponseNotFound	→ 404
HttpResponseForbidden	→ 403
HttpResponseNotAllowed	→ 405
HttpResponseGone	→ 410
HttpResponseServerError	→ 500

Si vous voulez relever une erreur 404 standard, vous pouvez lever une exception:

```
from django.http import Http404

def home(request, param=None):
    if request.GET:
        if 'name' in request.GET:
            string = request.GET['name']
            return HttpResponse("Bonjour %s!" % string)
        if 'test' in request.GET:
            raise Http404
    if param:
        return HttpResponse(f"<h1>Hello {param}</h1>")
    return HttpResponse("<h1>Hello Django</h1>")
```

L'erreur suivante devrait apparaître si vous ajouter une variable GET test dans votre url tel que :
<http://127.0.0.1:8000/monApp/home?test=cricri>

Page not found (404) Request Method: GET Request URL: http://127.0.0.1:8000/monapp/home?test=cricri Raised by: monapp.views.home

Comme nous l'avons vu la semaine dernière, l'url passe par une sorte de table de routage. L'url peut contenir un paramètre voire un id ;

```
path('home/<param>', views.home, name='home'),
```

Vous savez comment récupérer ce paramètre, peut importe son nom (param, id, pk) du moment qu'il est renseigné dans la vue.

```
def home(request, param=None):  
    if param:  
        return HttpResponse(f"<h1>Hello {param}</h1>")  
    return HttpResponse("<h1>Hello Django</h1>")
```

Comprendre les Templates

Nous allons commencer ce chapitre par la démonstration d'un anti-pattern. En programmation, un design pattern (patron de conception) est un style ou une technique qui représente la meilleure pratique et conduit à de bons résultats. À l'inverse, un anti-pattern représente une mauvaise pratique, quelque chose que nous devons éviter. Pour les besoins de la démonstration, la semaine dernière, certains ont pu écrire dans `monApp/views.py` leur vue `ListProduits`

```
def ListProduits(request):  
    prdts = Produit.objects.all()  
    chaine = "<ul>"  
    for prdt in prdts:  
        chaine += f"<li>{prdt.intituleProd}</li>"  
    chaine += "</ul>"  
  
    return HttpResponse(f"<h1>Liste de mes produits</h1>{chaine}")
```

Nous pourrions pousser le vice jusqu'à ajouter les éléments structurels d'une page web, tels que `<head>`, `<body>`, `<title>` et la balise `<html>`. Ce code fonctionne et fonctionnerait... mais c'est un anti-pattern !

Depuis le début... Pourquoi, quel est le problème avec cette approche ? Notre vue commence déjà à avoir l'air un peu chargée et la quantité de HTML ici ne fera que s'accroître au fur et à mesure de la construction de notre application. Le problème est que notre vue a maintenant deux responsabilités :

- sélectionner tous les objets `Produit` de la base de données : la logique métier;
- afficher les noms de ces produits parmi d'autres contenus comme les titres : la couche présentation.

Si une partie de notre application a trop de responsabilités, elle devient ingérable.

Alors, comment transformer cet anti-pattern en design pattern, et utiliser les meilleures pratiques ? Nous allons adhérer au principe de la responsabilité unique en déplaçant la responsabilité de la présentation hors de la vue et en la plaçant à sa place légitime : un gabarit ou **template**.

Commencez par créer un nouveau fichier à l'adresse : **monApp/templates/monApp/list_produits.html**. Notez la structure de répertoire que nous utilisons ici. Nous mettons toujours un sous-répertoire dans le répertoire des gabarits (**templates**) qui porte le même nom que l'application (**monApp**).

Vous pouvez aussi créer les templates à l'adresse **monProj/templates/monApp/** sous réserve de préciser dans le fichier **settings.py** le répertoire 'DIRS': [**BASE_DIR / 'templates'**], dans la variable **TEMPLATES**

Remplissons le fichier **list_produits.html** avec notre HTML :

```
<html>
  <head>
    <title>Mon app DJANGO</title>
  </head>
  <body>
    <h1>Hello mes produits !</h1>
    <p>Mes produits sont :</p>
    <!-- TODO : liste des produits -->
  </body>
</html>
```

Le code HTML n'est-il pas tellement plus beau dans un fichier .html que dans un fichier .py ? Vous bénéficiez d'une coloration syntaxique appropriée et d'une indentation automatique ! Mettons maintenant à jour notre vue **monApp/views.py**, de sorte qu'au lieu de définir son propre HTML, elle génère notre modèle à la place :

```
def ListProduits(request):
    prdts = Produit.objects.all()
    return render(request, 'monApp/list_produits.html')
```

Tout d'abord, nous importons la fonction **render**. Cet élément est probablement déjà présent puisqu'il est inclus dans le code de base, mais ajoutez-le si nécessaire. Dans la déclaration de retour, nous n'appelons plus le constructeur **HttpResponse**. Au lieu de cela, nous appelons la fonction **render** avec 2 arguments :

- L'objet **request** qui est passé dans la fonction ListProduits;
- Une chaîne de caractères contenant le chemin d'accès au fichier gabarit que nous avons créé.

Sous le capot, la fonction **render** crée un objet **HttpResponse** avec le HTML de notre modèle et le renvoie. Notre vue renvoie donc toujours une **HttpResponse** (ce qu'elle doit faire, pour être une vue). Jetez un coup d'œil à notre page dans le navigateur à l'url <http://127.0.0.1:8000/monApp/produits/>

C'est un bon début, mais les noms de nos produits n'apparaissent plus sur la page. Nous avons besoin d'un moyen d'injecter nos données de modèle dans notre gabarit. Il faut donc passer un objet contextuel au gabarit contenant une liste d'objets. Revenons à notre vue et ajoutons un troisième argument à notre appel de la méthode `render`. Cet argument doit être un dictionnaire python.

```
def ListProduits(request):  
    prdts = Produit.objects.all()  
    return render(request, 'monApp/list_produits.html',{'produits': prdts})
```

Ce dictionnaire est appelé dictionnaire contextuel. Chaque clé du dictionnaire devient une variable que nous pouvons utiliser dans notre gabarit, comme ceci :

```
<html>  
  <head>  
    <title>Mon app DJANGO</title>  
  </head>  
  <body>  
    <h1>Hello mes produits !</h1>  
    <p>Mes produits sont :</p>  
    <ul>  
      {% for prdt in produits %}  
        <li>{{ prdt.intituleProd }}</li>  
      {% endfor %}  
    </ul>  
  </body>  
</html>
```

Voici quelques éléments à prendre en compte :

- Les boucles et autres instructions logiques sont entourées de crochets et de signes de pourcentage (`{% ... %}`). Il s'agit de balises de gabarits.
- Une instruction `for` est construite en utilisant une syntaxe similaire à celle de Python, mais sans les deux-points de fin (`:`).
- Une balise de gabarit `for` doit posséder une balise de fermeture `endfor` plus loin dans le gabarit.
- L'espace entre les balises `for` et `endfor` peut contenir du texte, du HTML et même des variables de gabarits DJANGO.

Remarquez que les balises d'ouverture et de fermeture `<ul>` sont en dehors de la boucle : nous voulons qu'elles n'apparaissent qu'une seule fois. Les balises `<li>` sont à l'intérieur de la boucle, car nous avons besoin qu'elles soient répétées : une fois pour chaque groupe de la liste. Nous revoilà avec notre liste, mais maintenant nous la générons avec une boucle.

Parfois, lorsque nous injectons une variable dans notre gabarit, nous voudrions appliquer un certain formatage. Les filtres de gabarits peuvent nous y aider. Imaginons que nous voulions afficher les noms de nos produits dans une casse différente.

Nous pourrions utiliser le filtre `lower` (minuscules) ou `upper` (majuscules), comme ceci :

```
<li>{{ prdt.intituleProd | upper }}</li>
```

Vous appliquez un filtre à une variable en utilisant la barre verticale (`|`). Les filtres ne se limitent pas à la mise en forme du texte. Voyons comment nous pourrions afficher le nombre de groupes dans notre liste, en appliquant le filtre `length` à notre liste de groupes :

```
<p>J'ai {{ produits | length }} produits préférés.</p>
```

Nous pouvons également utiliser une logique d'embranchements dans nos gabarits. Dans notre gabarit, sous notre liste de groupes, ajoutons le code suivant :

```
<p>
    J'ai
        {% if produits|length < 5 %}
    peu de
        {% elif produits|length < 10 %}
    quelques
        {% else %}
    beaucoup de
        {% endif %}
    produits préférés.
</p>
```

Les instructions `if`, `elif` et `else` sont également semblables à la façon dont nous les écrivons en Python, mais là encore, nous omettons les deux-points de fin (`:`)

C'est à vous de jouer !

Maintenant que vous avez compris l'importance des gabarits et que vous avez vu certaines des choses utiles qu'ils permettent de faire, il est temps pour vous de créer vos propres gabarits :

- pour les vues `about`, `home` et `contact`,
- les vues `ListCategories`, `ListStatuts` et `ListRayons`.

Vos gabarits doivent être des pages HTML complètes, comprenant les balises `<html>`, `<head>`, `<title>` et `<body>`. Pour les vues listings, vous devez injecter vos objets dans le gabarit.

Maintenez votre taux de couverture de tests après ces modifications.

Si vous avez besoin d'indications, rappelez-vous que vous devez effectuer les opérations suivantes pour chaque gabarit que vous créez :

- Créer un fichier gabarit dans «`monApp/templates/monApp/`» et lui donner l'extension «`.html`».
- Déplacer votre HTML hors de la vue, et dans le gabarit.
- Changer la déclaration de retour de la vue pour appeler la méthode `render` et lui passer le chemin de votre fichier de gabarit.
- Passer également un dictionnaire de contexte à la méthode `render`.
- Utiliser des variables de gabarits pour injecter des données dans votre gabarit.
- Utiliser les balises de gabarits pour utiliser les boucles dans votre gabarit si besoin.

Héritage de templates

Avez-vous remarqué que nous avons maintenant du code répété dans nos fichiers de gabarits ? Cela deviendra ingérable à mesure que nous ajouterons des gabarits à notre site : si nous voulons modifier la balise `head`, nous devons la modifier dans chaque gabarit.

Voyons comment nous pouvons appliquer le principe DRY : Don't Repeat Yourself (Ne vous répétez pas), et éviter la répétition du code dans nos gabarits en utilisant un modèle de base.

Pour commencer, nous devons examiner nos gabarits existants et nous poser deux questions :

- Quelles sont les différences entre ces gabarits ?
- Qu'est-ce qui ne change pas ?
-

Si nous regardons tous les gabarits côte à côte, nous pouvons voir que les éléments communs aux trois sont :

- Les balises `<html>`
- Les balises `<head>`
- Les balises `<title>`
- Les balises `<body>`

Comme ces balises sont communes à toutes les pages, nous allons les définir à un seul endroit : notre gabarit de base. Plaçons ce code HTML pour l'ensemble du site dans un nouveau fichier gabarit à l'adresse «[monApp/templates/monApp/base.html](#)» :

```
<html>
  <head>
    <title>Mon app DJANGO</title>
  </head>
  <body>
    {% block contenu %}{% endblock %}
  </body>
</html>
```

Dans notre HTML, nous avons ajouté une balise de gabarit Django : la balise `block`. Et nous l'avons fermé avec une balise `endblock`. Nous avons également donné un nom à ce bloc : nous l'avons appelé `contenu`. Nous pouvons considérer la balise `block` comme un espace réservé, dans lequel nous pouvons injecter du contenu ; en ce sens, il est similaire aux variables de gabarits.

Ce que nous voulons faire ensuite, c'est injecter notre HTML spécifique à la page dans ce bloc. Revenons à nos gabarits de page pour voir comment on fait.

Ouvrons notre gabarit «`list_Produits.html`». Pour commencer, supprimons tout le HTML que nous définissons maintenant dans notre gabarit de base. On devrait aboutir à quelque chose comme ça :

```
<h1>Hello les produits !</h1>
<p>
    J'ai
    {% if produits|length < 5 %}
        peu de
    {% elif produits|length < 10 %}
        quelques
    {% else %}
        beaucoup de
    {% endif %}
    produits préférés.
</p>
<p>Mes produits préférés sont :</p>
<ul>
    {% for prdt in produits %}
        <li>{{ prdt.intituleProd | upper }}</li>
    {% endfor %}
</ul>
```

Maintenant notre gabarit de page ne contient que le contenu spécifique à cette page. Ensuite, ajoutons ces balises de gabarits supplémentaires en haut et en bas du gabarit

```
{% extends 'monApp/base.html' %}

{% block contenu %}

    <h1>Hello Django !</h1>
    ...
</ul>

{% endblock %}
```

La balise de gabarits `extends` en haut indique à DJANGO que nous voulons que ce gabarit hérite de notre gabarit de base. Autour de notre contenu, nous avons une balise d'ouverture et de fermeture `block` et nous avons donné à la balise d'ouverture le même nom que dans notre gabarit de base : `contenu`.

Comme vous l'avez peut-être deviné, DJANGO va prendre tout ce qui se trouve dans le bloc nommé `contenu` dans notre gabarit de page et l'injecter dans le bloc du même nom dans notre gabarit de base.

Voyons maintenant comment cela fonctionne dans notre navigateur. Votre page devrait apparaître exactement comme elle était avant. Nous n'avons pas modifié l'apparence, mais nous avons changé la façon dont les choses fonctionnent sous le capot. Cela rendra notre application plus facile à gérer au fur et à mesure de son développement.

Nous avons vu deux nouvelles balises de gabarits dans ce chapitre : `extends` et `block`, qui nécessitent qu'on leur passe un argument. C'est un peu comme lorsque nous passons un argument à une fonction Python, comme `print('Hello World!')`. Les arguments des balises de gabarits n'ont pas besoin d'être mis entre parenthèses, ils doivent simplement être séparés par des espaces.

Parfois, l'argument doit être une chaîne de caractères placée entre guillemets (par exemple : `{% extends "monApp/base.html" %}`), et parfois non (par exemple : `{% block content %}`), veuillez donc à utiliser la syntaxe appropriée pour la balise en question.

Bien entendu, maintenant que nous avons défini notre gabarit de base, vous pouvez l'utiliser pour toutes les pages existantes (et futures) de notre site !! C'est à vous de jouer !!! Je ne veux plus voir de HTML dans les vues...

Donnez du style à votre site grâce aux CSS

La balise `<head>` de notre site étant définie dans notre gabarit de base, c'est le bon moment pour ajouter une feuille de style. Les fichiers tels que les feuilles de style CSS sont appelés fichiers statiques dans DJANGO, car une fois l'application en cours d'exécution, ils ne changent pas. Nous plaçons les fichiers statiques à un endroit spécifique de notre application. Créez un dossier dans `monProj/static/css/` et à l'intérieur, créez un nouveau fichier appelé `styles.css` :

```
h1{ color: red ;}
```

Il ne s'agit pas de la feuille de style finale que nous utiliserons réellement dans notre application, c'est plutôt le « Hello World ! » des feuilles de style. Cela va donner à tous les titres `h1` de nos pages une couleur rouge. C'est juste pour tester que notre feuille de style se charge correctement ! Ouvrons le fichier `base.html` et ajoutons une balise `<link>` , sous le titre, pour charger notre feuille de style :

```
<html>
  <head>
    <title>Mon app DJANGO</title>
    <link rel="stylesheet" href="{% static 'css/styles.css' %}" />
  </head>
  <body>
    {% block contenu %}{% endblock %}
  </body>
</html>
```

Que remarquez-vous à propos de l'attribut `href` de notre balise `<link>` ?

Il s'agit en fait d'une autre balise de gabarits ! Cette balise `static` indique à DJANGO qu'il doit regarder dans le répertoire spécial `static` que nous avons créé précédemment et essayer de trouver le fichier `styles.css` que nous avons créé.

Maintenant, pour que la balise `static` fonctionne, nous devons d'abord la «charger» dans ce modèle. Nous le faisons en ajoutant une balise `load` au tout début du fichier, comme ceci :

```
{% load static %}
<html>
  <head>
    <title>Mon app DJANGO</title>
    <link rel="stylesheet" href="{% static 'css/styles.css' %}" />
  </head>
  <body>
    {% block contenu %}{% endblock %}
  </body>
</html>
```

Vous devriez voir vos titres en rouge. Cela signifie que tout fonctionne !

Si vous ne voyez pas de titre en rouge, essayez d'arrêter et de redémarrer le serveur de développement. Cela donnera à DJANGO une chance de trouver votre nouveau répertoire statique. Une fois que vous êtes sûr que votre fichier CSS se charge correctement, vous pouvez le modifier et commencer à donner à votre site le style que vous souhaitez. Mais nous pouvons mieux faire et nous passer du répertoire `static` pour ajouter du style...Installons Bootstrap avec cette commande :

```
pip install django-bootstrap-v5
```

Vous allez pouvoir ainsi bénéficier de quelques fonctionnalités de Bootstrap notamment. Pour cela, vous devez inclure le module `bootstrap5` dans le fichier `monProj/settings.py` :

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'monApp',
    'debug_toolbar',
    'bootstrap5',
]
```

Le fichier `base.html` devient par exemple

```
<!doctype html>
{% load static %}
<html lang="fr">
  <head>
    <meta charset="utf -8">
    <title>Mon app DJANGO</title>

    {% block styles %}
      <link rel="stylesheet" href="{% static 'css/styles.css' %}" />
      {% load bootstrap5 %}
      {% bootstrap_css %}
      {% bootstrap_javascript %}
    {% endblock %}
  </head>

  <body>
    {% block nav %}
      <nav class="navbar navbar-expand-md navbar-dark bg-primary mb-4">
        {% block menu %}{% endblock %}
      </nav >
    {% endblock %}

    <main role="main" class="container top-pushed">
      <div class="jumbotron">
        {% block contenu %}
        {% endblock %}
      </div>
    </main >

    <footer class="footer">
      <div class="container">
        <p>&copy; Copyright {% now "Y" %} by Titof Production Copyrigh (c)</p>
      </div>
    </footer>
  </body>
</html>
```

Notez les nouveaux blocs `styles`, `nav` et `menu`, les notions liées au chargement de Bootstrap, un container principal et un petit footer.

Notre fichier `list_produits.html` change lui aussi un peu :

```
{% extends 'monApp/base.html' %}

{% block menu %}
    <h1>Liste de mes produits !</h1>
{% endblock %}

{% block contenu %}
    <table class="table">
        <thead>
            <th>Id</th>
            <th>Nom produit</th>
            <th>Prix unitaire</th>
            <th>Date fabrication</th>
            <th>Catégorie</th>
            <th>Statut</th>
        </thead>
        <tbody>
            {% for prdt in produits %}
            <tr>
                <td>{{ prdt.refProd }} </td>
                <td>{{ prdt.intituleProd }} </td>
                <td>{{ prdt.prixUnitaireProd }} </td>
                <td>{{ prdt.dateFabProd }} </td>
                <td>{{ prdt.categorie }} </td>
                <td>{{ prdt.statut }} </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
    <p>
        J'ai
        {% if prdts|length < 5 %}
        peu de
        {% elif prdts|length < 10 %}
        quelques
        {% else %}
        beaucoup de
        {% endif %}
        produits préférés.
    </p>
{% endblock %}
```

Nous avons déjà une liste mieux présentée. Vous pouvez améliorer encore un peu la présentation en utilisant d'autres styles de bootstrap ...

C'est à vous de jouer !

Faites de même avec toutes vos listes (categorie, statut et rayon)